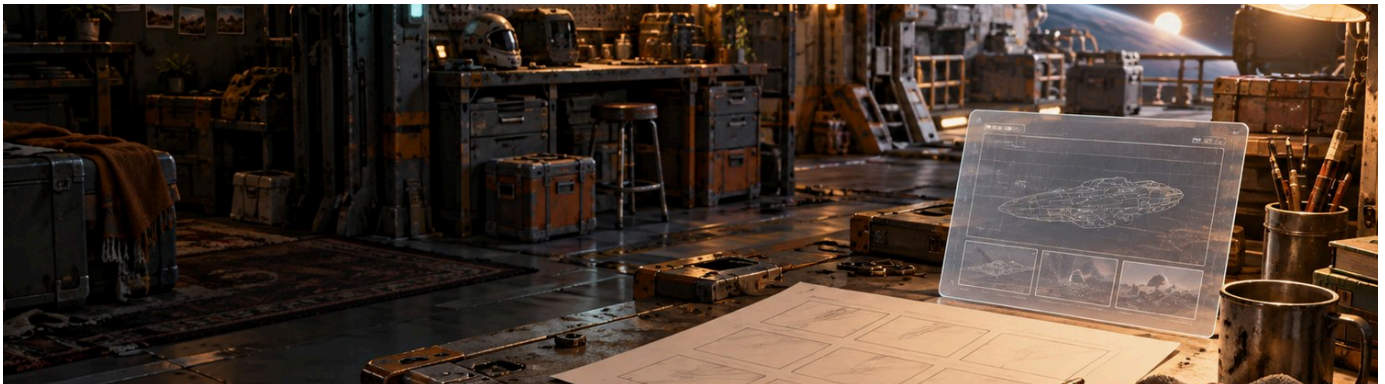
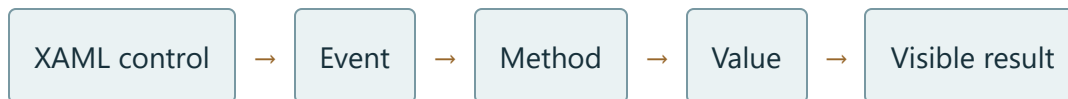


Know what you are building.



Read one concept when you reach it in your app. Explain it using your own controls, values and screenshots. The analogy helps you start; the definition tells you what C# actually does.



Change hats deliberately

Logical: What should happen? Draw input → check → action → result. Name the method, decision, repeat and stopping rule.

Visual: What should the user notice? Prototype the ready, result and recoverable-error states in PowerPoint. Show labels, imagery and the first action.

Data: What connects the screen and behavior? Name each value, its type, source and destination. Map one control to a variable, method and displayed result.

This is one project. The hats guide attention; they are not separate applications or a requirement for three people.

A DESIGN AND THE UNITS MADE FROM IT

Classes and objects

A class describes structure and behavior. An object is an instance of that class. `MainWindow` inherits `WPF Window`; a `Button` or `TextBox` in XAML creates a control object. You do not need to write a separate class for every button.

Window class → MainWindow class → your running window

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

A LABELED PLACE FOR A VALUE

Variables and types

A variable has a name, type and current value. A string holds text; `int` holds whole numbers; `bool` is true or false. `NameInput.Text` is a property on the `TextBox` object. A value can change while its name stays the same.

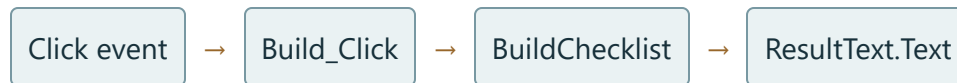
`NameInput.Text` → `userName (string)` → method parameter

```
string userName = NameInput.Text;
int itemCount = 3;
bool ready = itemCount > 0;
```

A NAMED TASK AND THE SIGNAL THAT REQUESTS IT

Methods and events

A method defines a task. Calling it performs that task. Parameters accept inputs; return supplies an output and exits. void means no value is returned. A Button Click event invokes its connected handler; the handler can call your smaller method. Functions and procedures are commonly expressed as methods in C#.

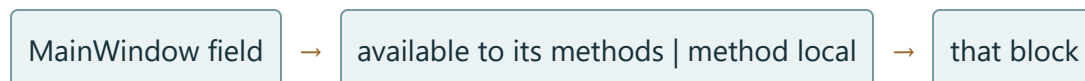


```
private string MakeGreeting(string name)
{
    return "Hello, " + name;
}
// Call from a handler:
ResultText.Text = MakeGreeting(NameInput.Text);
```

WHICH WORK AREA CAN USE THIS NAME?

Scope

A local variable can be used inside its declaring block, subject to declaration rules. A field belongs to a class or instance and can be used by its methods. The parameter userName belongs to one method call. Scope is name visibility, not security. Prefer a local variable unless the app needs to keep state between events.

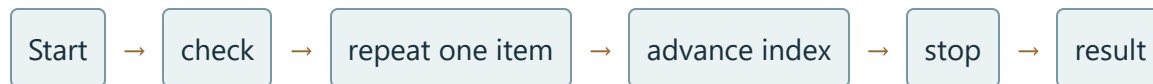


```
private int clicks = 0; // field inside MainWindow
private void CountClick()
{
    int amount = 1; // local variable
    clicks += amount;
}
```

A DECISION GATE AND A REPEAT WITH AN EXIT

Conditions and loops

if chooses a branch from a Boolean expression. for repeats while its condition is true. Change the counter and specify a bound. An unbounded loop on the UI thread can freeze the window. Small examples here use finite arrays and finish quickly.

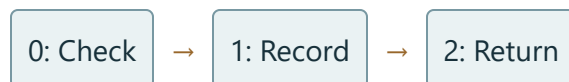


```
if (string.IsNullOrEmpty(userName))
{
    return "Enter a name.";
}
for (int index = 0; index < tasks.Length; index++)
{
    // Process tasks[index].
}
```

A FIXED ROW OF NUMBERED LOCKERS

Arrays and data routes

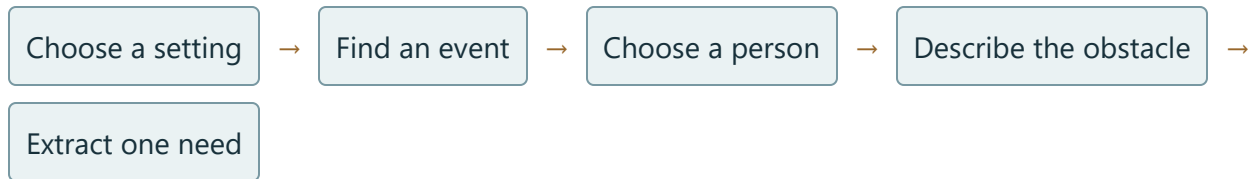
An array holds a fixed number of elements of the same type. The first index is 0; the last is Length - 1. Use index < Length, never <= Length. A tiny array introduces grouped data without a database. Trace where the values come from and where results appear.



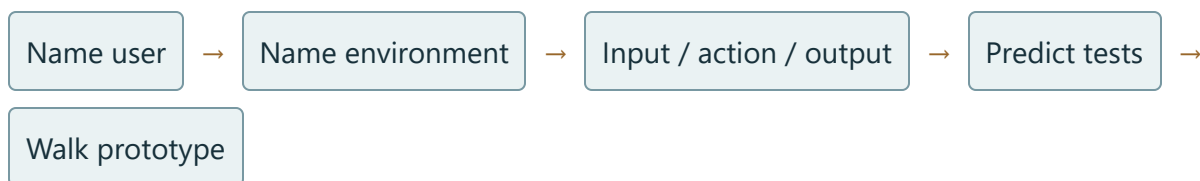
```
string[] tasks = { "Check", "Record", "Return" };
string firstTask = tasks[0];
int count = tasks.Length; // 3
```

Choose the next small move.

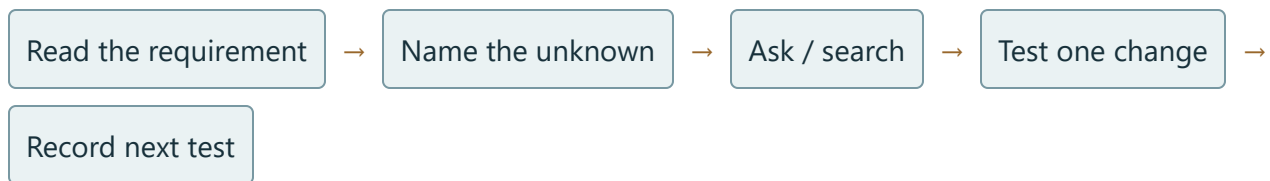
Story flow



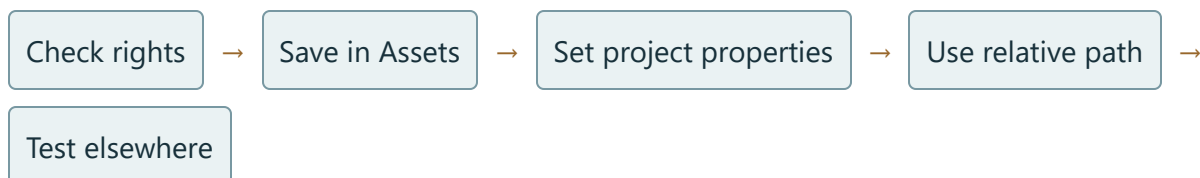
Before coding



When stuck



Media flow



Use owned, licensed or approved AI-generated media. Record the source/license or generation prompt. A reference screenshot does not license its artwork. Keep visible labels beside images and a no-sound route.

Make the manual useful

A reader should know where to start, which file to edit, what changes, why, and what result to expect. Include your own screenshots. Test at least one instruction with a classmate and revise it.

Companion examples are isolated unless identified as Quick Start edits. Do not paste every example into the same file. Return to [the Quick Start](#) for exact locations.